

第九章

(第 4、5、6、12、13、16 题)

4. 以下是在 IA-32+Linux 系统中执行的用户程序 P 的汇编代码:

```
1  # hello.s #
2  # display a string "Hello, world."
3
4  .section .rodata
5  msg:
6  .ascii "Hello, world.\n"
7
8  .section .text
9  .globl _start
10 _start:
11
12 movl $4, %eax      #系统调用号(sys_write)
13 movl $1, %ebx      #file descriptor (参数一):文件描述符(stdout)
14 movl $msg, %ecx    #string address (参数二):要显示的字符串
15 movl $14, %edx     #string length (参数三):字符串长度
16 int $0x80          #调用内核功能
17
18 movl $1, %eax      #系统调用号(sys_exit)
19 movl $0, %ebx      #参数一:退出代码
20 int $0x80          #调用内核功能
```

针对上述汇编代码, 回答下列问题。

- (1) 程序的功能是什么?
- (2) 执行到哪些指令时会发生从用户态转到内核态执行的情况?
- (3) 该用户程序调用了哪些系统调用?

参考答案:

- (1) 程序的功能是在标准输出设备(文件描述符为 `stdout`)上显示字符串“Hello, world.”。
- (2) 执行到第 16 行和第 20 行的“`int $0x80`”指令时会发生从用户态转到内核态执行的情况。
- (3) 该用户程序调用了 4 号系统调用 `write` 和 1 号系统调用 `exit`。

5. 第 4 题中用户程序的功能可以用以下 C 语言代码来实现:

```
1  int main()
2  {
3      write(1, "Hello, world.\n", 14);
4      exit(0);
5  }
```

针对上述 C 代码, 回答下列问题或完成下列任务。

- (1) 执行 `write()` 函数时, 传递给 `write()` 的实参在 `main` 栈帧中的存放情况怎样? 要求画图说明。
- (2) 从执行 `write()` 函数开始到调出 `write` 系统调用服务例程 `sys_write()` 执行的过程中, 其函数调用关系是怎样的? 要求画图说明。
- (3) 就程序设计的便捷性和灵活性以及程序执行性能等方面, 与第 4 题中的实现方式进行比较, 并说明哪个执行时间更短?

参考答案:

(1) 用“objdump -d”命令将对应的可执行文件反汇编后，其 main()函数对应的机器级指令序列如下图所示（左图是动态链接生成，右图是静态链接生成）。

<pre> 0004844c <main>: 0004844c: 55 push %ebp 0004844d: 89 e5 mov %esp,%ebp 0004844f: 83 e4 f0 and \$0xffffffff0,%esp 00048452: 83 ec 10 sub \$0x10,%esp 00048455: c7 44 24 08 0e 00 00 movl \$0xe,0x8(%esp) 0004845c: 00 0004845d: c7 44 24 04 10 85 04 movl \$0x8048510,0x4(%esp) 00048464: 08 00048465: c7 04 24 01 00 00 00 movl \$0x1,(%esp) 0004846c: e8 df fe ff ff call 8048350 <write@plt> 00048471: c7 04 24 00 00 00 00 movl \$0x0,(%esp) 00048478: e8 b3 fe ff ff call 8048330 <exit@plt> 0004847d: 90 nop 0004847e: 90 nop 0004847f: 90 nop </pre>	<pre> 00048254 <main>: 00048254: 55 push %ebp 00048255: 89 e5 mov %esp,%ebp 00048257: 83 e4 f0 and \$0xffffffff0,%esp 0004825a: 83 ec 10 sub \$0x10,%esp 0004825d: c7 44 24 08 0e 00 00 movl \$0xe,0x8(%esp) 00048264: 00 00048265: c7 44 24 04 48 a8 0a movl \$0x80aa848,0x4(%esp) 0004826c: 08 0004826d: c7 04 24 01 00 00 00 movl \$0x1,(%esp) 00048274: e8 57 76 00 00 call 804f8d0 <__libc_write> 00048279: c7 04 24 00 00 00 00 movl \$0x0,(%esp) 00048280: e8 cb 08 00 00 call 8048b50 <exit> </pre>
--	---

从上述指令序列可看出，在使用 call 指令调用 write()函数之前，main 函数在自身栈帧中传递了三个参数，首先在地址 R[esp]+8 处存放了 14 (0xe)，然后在 R[esp]+4 处存放了 0x8048510（动态链接）或 0x80aa848（静态链接），这个是指向字符串"Hello, world.\n"的指针，最后在 R[esp]处存放了 0x1。根据上述指令可以画出如右图所示的实参在 main 栈帧中的存放情况。



(2) 从执行 write()函数开始到调出 write 系统调用服务例程 sys_write()执行的过程中，首先通过 call 指令进入 write 封装函数执行。上述左图中的指令“call 8048350<write@plt>”（动态链接情况下）或右图中的指令“call 804f8d0<__libc_write>”（静态链接情况下）执行后，便进入封装函数 write 执行。

在进入 write()封装后，执行如下指令（静态链接情况下），可以看出，在执行陷阱指令（软中断指令）“int \$0x80”之前，有 4 条 MOV 指令，其中有三条 MOV 指令用来将 main 栈帧中的三个参数：字符串长度（0xe）、指向字符串"Hello, world.\n"的指针、文件描述符（0x1），分别送 EDX、ECX、EBX。还有一条 MOV 指令用来将调用号（4）送 EAX。

这样，当执行到完“int \$0x80”指令后，就从用户态陷入内核态，调出系统调用处理程序 system_call()执行。在 system_call()中，根据系统调用号为 4，再跳转到相应的系统调用服务例程 sys_write()执行，以完成将一个字符串写入文件的功能，其中，字符串的首地址由 ECX 指定，字符串的长度由 EDX 指出，写入文件的文件描述符由 EBX 指出。system_call()执行结束时，从内核返回的参数存放在 EAX 中。

```

0804f8d0 <__libc_write>:
804f8d0: 65 83 3d 0c 00 00 00    cmpl    $0x0,%gs:0xc
804f8d7: 00                                jne     804f8fb <__write_nocancel+0x21>
804f8d8: 75 21                                jne     804f8fb <__write_nocancel+0x21>

0804f8da <__write_nocancel>:
804f8da: 53                                push    %ebx
804f8db: 8b 54 24 10                    mov     0x10(%esp),%edx
804f8df: 8b 4c 24 0c                    mov     0xc(%esp),%ecx
804f8e3: 8b 5c 24 08                    mov     0x8(%esp),%ebx
804f8e7: b8 04 00 00 00                mov     $0x4,%eax
804f8ec: cd 80                          int     $0x80
804f8ee: 5b                                pop     %ebx
804f8ef: 3d 01 f0 ff ff                cmp     $0xfffff001,%eax
804f8f4: 0f 83 f6 1f 00 00            jae     80518f0 <__syscall_error>
804f8fa: c3                                ret
804f8fb: e8 f0 0d 00 00                call    80506f0 <__libc_enable_asynccancel>
804f900: 50                                push    %eax
804f901: 53                                push    %ebx
804f902: 8b 54 24 14                    mov     0x14(%esp),%edx
804f906: 8b 4c 24 10                    mov     0x10(%esp),%ecx
804f90a: 8b 5c 24 0c                    mov     0xc(%esp),%ebx
804f90e: b8 04 00 00 00                mov     $0x4,%eax
804f913: cd 80                          int     $0x80
804f915: 5b                                pop     %ebx

```

(3) 显然，对于第 3 题给出的实现方式，其程序设计的便捷性和灵活性都不如本题给出的实现方式，第 3 题采用汇编程序设计方式，只要参数不同，就需要重新编写不同的指令；而本题采用高级语言程序设计方式，只要在 `write` 函数中改变不同的实参，就可以完成不同的功能。

不过，第 3 题实现方式下的程序执行性能比本题实现方式好，因为用汇编实现时，省去了高级语言程序中大量的函数调用，因而它的执行时间更短。

6. 第 4 题和第 5 题中用户程序的功能可以用以下 C 语言代码来实现：

```

1  #include <stdio.h>
2  int main()
3  {
4      printf( "Hello, world.\n");
5  }

```

假定源程序文件名为 `hello.c`，可重定位目标文件名为 `hello.o`，可执行目标文件名为 `hello`，程序用 GCC 编译驱动程序处理，在 IA-32+Linux 系统中执行。回答下列问题或完成下列任务。

- (1) 为什么在 `hello.c` 的开头需加 “`#include <stdio.h>`”？为什么 `hello.c` 中没有定义 `printf()` 函数，也没它的原型声明，但 `main()` 函数引用它时没有发生错误？
- (2) 需要经过哪些步骤才能在机器上执行 `hello` 程序？要求详细说明各个环节的处理过程。
- (3) 为什么 `printf()` 函数中没有指定字符串的输出目的地，但执行 `hello` 程序后会在屏幕上显示字符串？
- (4) 字符串 “`Hello, world.\n`” 在机器中对应的 0/1 序列（机器码）是什么？这个 0/1 序列存放在 `hello.o` 文件的哪个节中？这个 0/1 序列在可执行目标文件 `hello` 的哪个段中？
- (5) 若采用静态链接，则需要用到 `printf.o` 模块来解析 `hello.o` 中的外部引用符号 `printf`，`printf.o` 模块在哪个静态库中？静态链接后，`printf.o` 中的代码部分（.text 节）被映射到虚拟地址空间的哪个段中？若采用动态链接，则函数 `printf()` 的代码在虚拟地址空间中的何处？
- (6) 假定 `printf()` 函数最终调用的 `write` 系统调用的封装函数为 `write()`，其对应的汇编代码如下：

```

804f8fa: 53                                push    %ebx
804f8fb: 8b 54 24 10                    mov     0x10(%esp), %edx
804f8ff: 8b 4c 24 0c                    mov     0xc(%esp), %ecx
804f903: 8b 5c 24 08                    mov     0x8(%esp), %ebx
804f907: b8 04 00 00 00                mov     $0x4, %eax
804f90c: cd 80                          int     $0x80

```

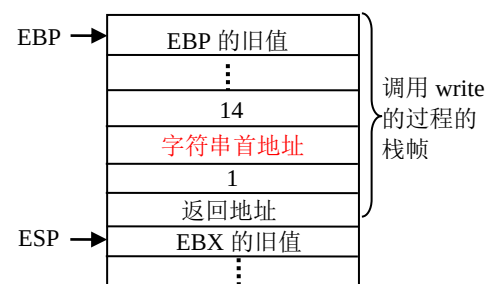
804f90e:	5b	pop	%ebx
804f90f:	3d 01 f0 ff ff	cmp	\$0xfffff001, %eax
804f914:	0f 83 f6 1f 00 00	jae	8051910 <__syscall_error>
804f91a:	c3	ret	

请给出以上每条汇编指令的注释，并说明该 Linux 系统中系统调用返回的最大错误号是多少？

- (7) 就程序设计的便捷性和灵活性以及程序执行性能等方面，与第 4 题和第 5 题中的实现方式分别进行比较，并分析说明哪个执行时间更短？

参考答案：

- (1) 因为 hello.c 中使用了 C 库函数 printf(), 所以需在 hello.c 文件的开头加 “#include <stdio.h>”。因为在 stdio.h 头文件中有 printf() 函数的原型声明，并且 printf() 函数是 C 库函数，所以，虽然 hello.c 中没有定义 printf() 函数，也没它的原型声明，但是，通过对 hello.c 和 stdio.h 的预处理，编译器会得到 printf() 的原型声明，从而获得符号 printf 的相关信息，使得链接器进行链接的时候，能够从标准 C 库 (libc.a 或 libc.so) 中得到 printf 模块的信息，完成链接工作。
- (2) 需要经过预处理、编译、汇编、链接才能生成可执行文件 hello，然后通过启动 hello 程序执行。预处理阶段主要是对带 # 的语句进行处理；编译阶段主要是将预处理后的源程序文件编译生成汇编语言程序；汇编阶段主要是将汇编语言源程序转换为可重定位的机器语言目标代码文件；链接阶段将多个可重定位的机器语言目标代码以及库函数链接起来，生成最终的可执行文件。
- (3) 因为 printf() 函数默认的输出设备为标准输出设备 stdout，所以无需指定字符串的输出目的地。执行 hello 程序后，自动会在 stdout 设备（屏幕上）显示字符串。
- (4) 字符串 “Hello, world.\n” 在机器中对应的 0/1 序列（机器码）是该字符串中每个字符对应的 ASCII 码。即 “0100 1000 0110 0101”。这个 0/1 序列存放在 hello.o 文件的 .rodata 节中，作为只读数据使用，所以，从汇编指令（如 “movl \$0x8048510, 0x4(%esp)” 指令）中可以看到，字符串的首地址是一个绝对地址（如 0x8048510），这个 0/1 序列在可执行目标文件 hello 的只读代码段（read-only code segment）中。
- (5) 若采用静态链接，则需要用到 printf.o 模块来解析 hello.o 中的外部引用符号 printf，printf.o 模块在静态库 libc.a 中。静态链接后，printf.o 中的代码部分（.text 节）被映射到虚拟地址空间的只读代码段（read-only code segment）中。若采用动态链接，则函数 printf() 的代码在虚拟地址空间中的共享库映射区。
- (6) write() 函数的调用语句为 “write(1, “Hello, world.\n”, 14);”，参数按照从右向左的顺序压栈，即 14 先压栈，然后是字符串的首地址压栈，最后是 fd=1 压栈。随后执行 call 指令，将返回地址压栈后跳转到 write 代码执行。在 write 代码中，第一条指令将 EBX 压栈。因此，此时栈中的状态如右图所示。可以看出，在地址 R[esp]+8 处存放了 fd=1，在 R[esp]+12 处存放了指向字符串 “Hello, world.\n” 的指针，在 R[esp]+16 处存放了 14。汇编指令的注释如下。



804f8fa:	53	push	%ebx	//被调用者保存寄存器 EBX 入栈
804f8fb:	8b 54 24 10	mov	0x10(%esp), %edx	//将字符串长度 14 送 EDX
804f8ff:	8b 4c 24 0c	mov	0xc(%esp), %ecx	//将字符串首地址送 ECX
804f903:	8b 5c 24 08	mov	0x8(%esp), %ebx	//将文件描述符 fd=1 送 EBX
804f907:	b8 04 00 00 00	mov	\$0x4, %eax	//将调用号 4 送 EAX
804f90c:	cd 80	int	\$0x80	//系统调用入口
804f90e:	5d	pop	%ebx	//恢复 EBX 的旧值

```

804f90f: 3d 01 f0 ff ff      cmp     $0xffff001, %eax    //将系统调用返回值与 0xffff001 比较
804f914: 0f 83 f6 1f 00 00   jae     8051910 <__syscall_error> //大于等于时转出错处理
804f91a: c3                  ret                     //返回到调用 write 的过程

```

从上述代码可以看出，该 Linux 系统中系统调用返回的最大错误号是 4095。因为当返回值大于等于 0xffff001 时进行出错处理，显然，这些值在 0xffff001 (-4095) ~ 0xffffffff (-1) 之间，通常错误号是正整数，因此需要对其取负，因此，错误号范围为 4095~1。

- (7) 显然，第 4 题和第 5 题给出的实现方式，其程序设计的便捷性和可移植性都不如本题给出的实现方式，第 4 题采用汇编程序设计方式，只要参数不同，就需要重新编写不同的指令；第 5 题采用 write 函数直接进行系统调用，只能在支持 write 系统调用的系统（类 UNIX）中执行。而本题给出的是 C 库函数调用，所以，可以在不同的系统中执行，只要该系统具有 C 语言程序设计环境即可，因而本题给出的程序的可移植性更好。

不过，第 4 题实现方式下的程序执行性能最好，因为用汇编实现时，省去了高级语言程序中大量的函数调用，因而它的执行时间最短。

12. 某台打印机每分钟最快打印 6 个页面，页面规格为 50 行×80 字符。已知某计算机主频为 500MHz，若采用中断方式进行字符打印，则每个字符申请一次中断且中断响应和中断处理时间合起来为 1000 个时钟周期。请问该计算机系统能否采用中断控制 I/O 方式来进行字符打印输出？为什么？

参考答案：

要达到打印机最快的打印速度，打印机的数据传输率应达到每分钟为 $6 \times 50 \text{ 行} \times 80 \text{ 字符} = 24000 \text{ 字符}$ ，即打印机的数据传输率应为 $24000 \text{ 字符} / 60 \text{ 秒} = 400 \text{ 字符/s}$ 。

若采用中断方式打印字符，则大约每隔 $1/400 \text{ 秒} = 2.5\text{ms}$ 发出一次中断申请。

实际的中断响应和中断处理时间为 $1000 \times 1/500\text{M} \times 1000 = 0.002\text{ms}$ 。

因为 $0.002\text{ms} \ll 2.5\text{ms}$ ，所以可以采用中断方式进行字符打印输出。

13. 假定某计算机的 CPU 主频为 500MHz，所连接的某个外设的最大数据传输率为 20kB/s，该外设接口中有一个 16 位的数据缓存器，相应的中断服务程序的执行时间为 500 个时钟周期，则是否可以用中断方式进行该外设的输入输出？假定该外设的最大数据传输率改为 2MB/s，则是否可以用中断方式进行该外设的输入输出？

参考答案：

因为该外设接口中有一个 16 位数据缓存器，所以，若用中断方式进行输入/输出，则可以每 16 位数据进行一次中断请求，因此，中断请求的时间间隔为 $10^6 \times 2\text{B} / 20\text{kB} = 100\mu\text{s}$ 。对应的中断服务程序的执行时间为 $(10^6 / 500\text{M}) \times 500 = 1\mu\text{s}$ ，因为中断响应过程就是执行一条隐指令的过程，所用时间相对于中断处理时间（即执行中断服务程序的时间）而言，几乎可以忽略不计，因而整个中断响应并处理的时间大约 $1\mu\text{s}$ 多一点，远远小于中断请求的间隔时间。因此，可以用中断方式进行该外设的输入输出。若用中断方式进行该设备的输入/输出，则该设备持续工作期间，CPU 用于该设备进行输入/输出的时间占整个 CPU 时间的百分比大约为 $1/100 = 1\%$ （也可以通过考察 1 秒钟内 500M 个时钟周期中有多少时钟周期用于中断来计算百分比，其计算公式为 $(10^6 / 100 \times 500) / 500\text{M} = 1\%$ ）。

若外设的最大传输率为 2MB/s，则中断请求的时间间隔为 $10^6 \times 2\text{B} / 2\text{MB} = 1\mu\text{s}$ 。而整个中断响应并处理的时间大约 $1\mu\text{s}$ 多一点，中断请求的间隔时间小于中断响应和处理时间，也即中断处理还未结束就会有该外设新的中断到来，所以不可以用中断方式进行该外设的输入输出。

16. 假设某计算机所有指令都在两个总线周期内完成，一个总线周期用来取指令，另一个总线周期用来存取数据。总线周期为 250ns，因而每条指令的执行时间为 500ns。若该计算机中配置的磁盘上每个磁

道有 16 个 512 字节的扇区，磁盘旋转一圈的时间是 8.192ms，总线宽度 16 位，采用 DMA 方式传送磁盘数据，则在进行 DMA 传送时该计算机指令执行速度降低了百分之几？

参考答案：

磁盘的平均数据传输率为 $10^3 \times 16 \times 512\text{B} / 8.192 = 1\text{MB/s}$ 。总线位宽为 16 位，DMA 控制器每隔 $2\text{B} / (1\text{MB/s}) = 2\mu\text{s}$ 申请一次数据传送，在 $2\mu\text{s}$ 期间 CPU 共执行 $2\mu\text{s} / 500\text{ns} = 4$ 条指令，因此，每 4 条指令的执行被插入一个总线周期，用于一次数据传送，也即平均每条指令延长了 $250/4 = 62.5\text{ns}$ 。因而，由于采用 DMA 方式，计算机执行指令的速度降低了大约 $62.5/500 = 12.5\%$ 。